mybibli

User Manual

The mybibli authors
https://github.com/guycorbaz/mybibli

Version 1.1.7 — May 17, 2026

This manual covers mybibli version 1.1.7 and later (the v1.1.0 floor is still enforced for fresh installs — see the Installation chapter for the seed-gate background).

Contents

Preface

Who this manual is for

This manual is written for the person who installs, configures and runs a mybibli instance for their household, a small association, or a community library. It assumes you can:

- open a terminal and run `docker compose` commands;
- edit a plain-text configuration file with the editor of your choice;
- read a URL printed in a log line and open it in a browser.

No Rust, MariaDB or HTMX knowledge is required. The manual is deliberately scoped to the *operator* role — building mybibli from source or modifying its code is covered by `CLAUDE.md` and the planning artifacts in the repository.

Conventions

Commands. Shell snippets are shown in a fixed-width box:

```
docker compose up -d
```

Configuration files. File excerpts use the same style with the file name called out in the surrounding text. Comments inside the snippet use the file's native comment marker.

Callouts. Three sidebars highlight important text:

Note

A *note* provides background or context that helps you understand why something works the way it does. Skipping it is safe.

Tip

A *tip* suggests a practice that makes day-to-day use smoother. Try it once you are comfortable with the basics.

Warning

A *warning* flags a step where a mistake can lose data, lock you out of the admin panel, or expose the instance to unauthorized access. Read these in full.

Cross-references. Chapters and sections are linked in the PDF — click any blue title or page number to jump there. The index at the end maps every important term to its page of first use.

Getting help

The canonical source of truth for mybibli is the GitHub repository:

https://github.com/guycorbaz/mybibli

Open an issue against the `guycorbaz/mybibli` repository when you hit a bug, a documentation gap, or a missing feature. The issue templates ask for the version (visible at the bottom of any page in the admin panel), your install method, and the steps to reproduce.

Manual coverage

This manual covers mybibli **1.1.0 and later**. Older versions shipped a security-relevant bug where the first-launch setup wizard was silently bypassed — see chapter 8 for the upgrade path from 1.0.x.

Happy cataloging.

Chapter 1

Installation

This chapter walks you through installing mybibli on your own hardware. The reference deployment target is a home server, NAS, or any always-on Linux box that can run Docker.

1.1 System requirements

- **Operating system:** any Linux distribution, macOS, or Windows host capable of running Docker. The host kernel must support cgroups v2 (any distribution from 2021 onward).
- **Docker:** version 24 or later, with the `compose` plugin. Test with `docker compose version`.
- **Disk:** at least 1 GB free for the application image plus the database. Cover images add roughly 50 KB per title.
- **Memory:** 512 MB is enough for a household-sized library (a few thousand titles); 1 GB gives generous headroom for the metadata-fetch pipeline.
- **Network:** outbound HTTPS to the metadata providers you choose to use (see chapter 5). LAN-only deployments work without any inbound port forwarding.

Warning

Do not install mybibli versions below 1.1.0. Every published image with a tag below `1.1.0` contains a seed migration that creates the credentials `admin/admin` and `librarian/librarian` on *every* fresh install, including production. The first-launch setup wizard is silently bypassed. Use 1.1.0 or later; the pre-1.1.0 images will be removed from Docker Hub to prevent accidental use.

1.2 Pre-installation checklist

Before you start:

1. Pick a host directory for the mybibli files (e.g. `/srv/mybibli` on Linux, `/volume1/docker/mybibli` on Synology DSM).
2. Decide how cover images are stored. The shipped `docker-compose.yml` declares a Docker *named volume* (`mybibli_covers`) — zero configuration, Docker manages persistence and backup via `docker volume`. If you prefer a host-visible directory (Synology DSM file

browser, manual rsync backups), pick a host path now (e.g. `/volume1/docker/mybibli/covers`) and you will switch the compose file to a bind mount in step 3 below.

3. Choose a strong password for the bundled MariaDB `root` and application users — or, for an external database, create a dedicated database and user up front (see section 1.4).

1.3 Method 1 — Bundled database

This is the simplest path. The shipped `docker-compose.yml` file declares two services: the mybibli application and a MariaDB instance pinned to a known version. Both are managed as one unit.

1. Get the compose file and `.env`

Download the `docker-compose.yml` and `.env.example` from the release you intend to install (or check out the repository at the matching tag):

```
mkdir -p /srv/mybibli && cd /srv/mybibli
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.1.0/docker-compose.yml
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.1.0/.env.example
cp .env.example .env
```

1b. (Optional) Switch covers to a bind mount

By default, the shipped `docker-compose.yml` stores downloaded cover images in the Docker-managed named volume `mybibli_covers`. This works zero-config and persists across container upgrades (issue #213). Most operators should leave it as-is.

If you want covers visible in your host file manager (Synology DSM File Station, manual rsync backups, etc.), edit the compose file:

- In the `mybibli` service's `volumes:` block, comment out the `- mybibli_covers:/app/covers` line and uncomment the `- ${COVERS_HOST_PATH:-./covers-host}:/app/covers` line right below it.
- In `.env`, set `COVERS_HOST_PATH` to the host path you picked in the pre-installation checklist (e.g. `COVERS_HOST_PATH=/volume1/docker/mybibli/covers`).
- Create the directory before starting the stack (`mkdir -p`) so Docker doesn't auto-create it owned by root.

2. Edit `.env`

Open `.env` in your editor. Every variable carries an explanatory comment; the values that matter for a bundled-database install are:

```
DATABASE_URL=mysql://mybibli:mybibli_password@db:3306/mybibli?charset=utf8mb4
MYSQL_HOST=db
MYSQL_DATABASE=mybibli
MYSQL_USER=mybibli
MYSQL_PASSWORD=<choose-a-strong-password>
MYSQL_ROOT_PASSWORD=<choose-a-strong-password>

HOST_PORT=8080         # change if 8080 is already taken on the host
APP_LANGUAGE=en        # or 'fr' for the French UI by default

MYBIBLI_COOKIE_SECURE=false   # set to true only behind HTTPS
```

The `DATABASE_URL` *must* reference the same user, password and database name as the `MYSQL_*` values — the compose file rebuilds the URL from those parts but the Rust binary reads `DATABASE_URL` directly, so the two must agree.

Warning

The default passwords in `.env.example` (`mybibli_password`, `root_password`) are placeholders, not credentials to keep. Change them before the first `docker compose up`. After the database has been created, rotating these passwords requires also rewriting the MariaDB user entries — easier to get it right the first time.

3. Start the stack

```
docker compose up -d
docker compose logs -f mybibli
```

The first launch:

1. pulls the `gcorbaz/mybibli:1.1.0` (or later) image and the MariaDB image;
2. starts the database and waits for it to be healthy;
3. runs the mybibli database migrations (one-time schema setup);
4. starts the HTTP listener on the port you configured.

You can stop tailing the logs with `Ctrl-C`; the containers keep running detached.

4. Open the setup wizard

Visit `http://<host>:8080/setup` in a browser. The first-launch setup wizard appears (covered in section 1.6).

1.4 Method 2 — External database (Synology NAS example)

If you already run MariaDB on your NAS — for example via the Synology **MariaDB 10** package — you can point mybibli at it instead of running a second database in Docker. The mybibli image is unchanged; only the compose file and `.env` differ.

1. Create the database and user

Connect to the external MariaDB as `root` and run:

```
CREATE DATABASE mybibli CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'mybibli'@'%' IDENTIFIED BY 'choose-a-strong-password';
GRANT ALL PRIVILEGES ON mybibli.* TO 'mybibli'@'%';
FLUSH PRIVILEGES;
```

Note

On Synology DSM, MariaDB 10 listens on port `3307` by default (not 3306 — that port is reserved for the older MySQL package). Open **MariaDB 10** → **Settings** and tick *Enable TCP/IP connection* so containers on the same Docker bridge can reach the server.

2. Trim the compose file

Edit `docker-compose.yml` and remove the bundled database:

- delete (or comment out) the entire `db:` service;
- delete (or comment out) the `depends_on:` block on the `mybibli` service;
- in the bottom `volumes:` block, remove `mybibli_db_data:` only — **keep** `mybibli_covers:`. Cover images are local to the app and must survive container upgrades regardless of where the database lives (issue #213).

3. Point `.env` at the external database

```
DATABASE_URL=mysql://mybibli:strong-password@host.docker.internal:3307/mybibli?charset=
    utf8mb4
```

Substitute the host name and port that match your network. From a mybibli container running on the same Docker bridge as the NAS host, `host.docker.internal` resolves to the host on macOS and recent Linux Docker installs; on older Docker engines, use the LAN IP of the NAS instead (e.g. `192.168.1.10`).

The `MYSQL_*` variables are now unused — leave them blank or delete them.

4. Start the application

```
docker compose up -d
docker compose logs -f mybibli
```

The first launch runs the schema migrations against the external database. The Synology MariaDB package will hold this schema for as long as the package exists; backing it up is covered by the NAS-level backup tools (see chapter 7).

Tip

The external-database setup is the recommended path when you already maintain a database server, because it gives you one backup pipeline to monitor instead of two. The bundled-database setup is faster to get going if you have no other workloads on the host.

1.5 Environment variable reference

The full reference lives in the file `.env.example` at the repository root — every variable carries an inline comment. The table below summarises which variable applies where.

All boolean variables follow a **strict accept-set**: only the strings `1`, `true` and `TRUE` count as "on". Anything else (including the empty string, `0`, and `false`) is treated as "off". This avoids the classic footgun where a stale shell value like `0` reads as "set" and silently flips an opt-out.

Variable	Default	Purpose
Database		
`DATABASE_URL`	— (required)	MariaDB connection string (DSN).
`MYSQL_HOST`	`db`	Hostname for the bundled DB service.
`MYSQL_PORT`	`3306`	Port for the bundled DB service.
`MYSQL_DATABASE`	`mybibli`	DB name for the bundled service.
`MYSQL_USER`	`mybibli`	DB user for the bundled service.
`MYSQL_PASSWORD`	—	DB user password for the bundled service.
`MYSQL_ROOT_PASSWORD`	—	Root password for the bundled service.
HTTP server		
`HOST`	`0.0.0.0`	Bind address inside the container.
`PORT`	`8080`	Bind port inside the container.
`HOST_PORT`	`8080`	Port published by Docker on the host.
Application		
`RUST_LOG`	`mybibli=info`	Tracing filter. Use `mybibli=debug` for verbose logs.
`APP_LANGUAGE`	`en`	Default UI language for anonymous visitors.
`COVERS_DIR`	`./covers`	Filesystem path for cover-image storage.
Hardening		
`MYBIBLI_COOKIE_SECURE`	`false`	Set to `true` only behind HTTPS.
`CSP_REPORT_ONLY`	`false`	Switch CSP header to report-only mode.
Metadata providers (migrated to DB on first boot)		
`GOOGLE_BOOKS_API_KEY`	—	Optional key for Google Books.
`OMDB_API_KEY`	—	Optional key for OMDb (films/series).
`TMDB_API_KEY`	—	Optional key for TMDb (films/series).
Optional dev / test overrides		
`MYBIBLI_SKIP_SETUP`	`false`	Bypass the first-launch wizard. *Never on prod.*
`MYBIBLI_SKIP_STARTUP_PURGE`	`false`	Skip the boot-time auto-purge.
`MYBIBLI_SEED_DEV_USERS`	`false`	Seed dev `admin/admin` + `librarian/librarian`. *Never on prod.*

Provider base-URL overrides (`BNF_API_BASE_URL`, `GOOGLE_BOOKS_API_BASE_URL`, …) exist solely to point the metadata pipeline at the in-tree mock server during automated tests. Leave them unset in production.

1.6 First boot — the setup wizard

On a clean install, every request is redirected to `/setup` until the wizard completes. The wizard has four steps:

1. **Admin account.** Choose a username and a strong password for the first administrator. This step is the whole reason the wizard exists — once the account is created, the wizard's gate predicate flips and subsequent visitors land on the public catalog instead.

2. **Metadata providers.** Paste in any API keys you have on hand (Google Books, OMDb, TMDb). All keyed providers are optional — you can skip this step entirely and add keys later from `/admin > System > Metadata Providers`.

3. **Preferences.** Pick the default UI language for anonymous visitors and the loan overdue threshold (in days).

4. **Done.** A summary screen. After confirming, the wizard writes `settings.setup_completed_at` and disables itself for the lifetime of the installation.

The wizard is idempotent: if you close the browser between steps, reopen `/setup` and the wizard resumes server-side at the right step (it queries the database on every load — there is no client state to lose).

1.7 Verifying the install

After the wizard finishes:

- Log in at `http://<host>:8080/login` with the admin credentials you just created.
- Visit `/admin > Health`. The page should show non-zero entity counts (zero on a clean install — that is expected) and a green check next to each metadata provider that has been reachable in the last 5 minutes.
- Scan an ISBN (the search field at the top of the home page) to confirm the metadata pipeline resolves a real title. The first scan takes a couple of seconds; the result appears in the timeline of recent additions on the home page once the background fetch completes.

If anything goes wrong, jump to chapter 10.

Chapter 2

Configuration

Once the setup wizard is behind you, the day-one configuration work consists of three blocks: defining your *storage locations*, populating the *reference data* (genres, contributor roles, volume states, location types), and adjusting the system-wide *settings* (overdue threshold, auto-purge cadence, provider keys).

Everything in this chapter is configured from the `/admin` panel — there is no separate configuration file. The panel has five tabs:

- **Health** — read-only status dashboard.
- **Users** — create / deactivate users, change roles.
- **Reference data** — the four taxonomy tables.
- **Trash** — view and restore soft-deleted items.
- **System** — global settings.

Only the `Admin` role sees the panel; `Librarian` and `Anonymous` users get a 403 from every `/admin/*` route.

2.1 Storage locations and hierarchy

Storage locations are the physical spots where your volumes live: a room, a shelf, a row, a numbered box, a specific cell in a cabinet. mybibli organises them as a **tree** — every location has at most one parent. The hierarchy is arbitrary: you decide both the levels and the labels.

A workable household hierarchy might be three levels deep:

```
Living room          (room)
   Bookshelf A      (shelf)
        Row 1       (row)
        Row 2       (row)
   Bookshelf B      (shelf)
        Row 1       (row)
Bedroom              (room)
   Nightstand      (shelf)
```

Three levels is enough to find any volume in seconds without forcing the hierarchy to mirror every nook of your living space. A flat list (one big "shelf" per physical shelf) works equally well for collections under a few hundred volumes.

Editing the tree

Open `/locations`. The tree is rendered as a nested list with inline create / rename / move / delete affordances. Drag-and-drop is *not* supported (mybibli ships with a CSP-strict no-JS policy); moves are done by editing a node and picking a new parent from a dropdown.

Location node types

Each location node carries a **node type** — "room", "shelf", "row", "box", anything you like. Node types are managed under `/admin > Reference data > Location node types`. They are pure labels; mybibli does not enforce that a "shelf" must sit under a "room".

> **Tip**
>
> Renaming a node type cascades to every location that uses it (the type is stored by name in the locations table). Deleting a node type is refused as long as one location still references it.

2.2 Labels: V-codes and L-codes

mybibli prints two kinds of barcode labels:

- **V-codes** — one per *volume* (the physical book on a shelf). Format: `V` followed by a zero-padded sequence (e.g. `V0042`).
- **L-codes** — one per *storage location*. Format: `L` followed by a zero-padded sequence (e.g. `L0007`).

V-codes and L-codes are scanned with the same barcode scanner you use for ISBNs at cataloging time. The scanner's prefix (`V` or `L`) tells mybibli which lookup to perform.

Printing labels

Open `/locations` or the relevant volume page and click **Print label**. mybibli renders a printable HTML page with the Code-128 barcode and human-readable text. Print on plain paper or sticker stock — the encoding is forgiving enough that a low-cost inkjet works for V-codes and L-codes alike.

> **Tip**
>
> Print V-code stickers *before* you start cataloging in bulk. A roll of pre-printed V0001 through V0500 means you can grab a sticker, stick it on the back cover, and scan in one motion — no context-switch between the cataloging UI and the printer.

2.3 Hardware: barcode scanner

mybibli is designed around a barcode scanner but does not strictly require one — every ISBN, V-code and L-code field accepts manual keyboard input as a fallback. That said, for any collection beyond a handful of titles, a cheap USB scanner pays for itself within the first dozen scans.

What to buy

Any *HID-keyboard-mode* USB scanner in the 15–30 € range works. The HID mode is what makes the scanner driver-free: the OS sees it as a regular keyboard and the scanned digits land directly in whatever input field has focus. Almost every general-purpose USB scanner on the market ships in this mode by default; look for the words "HID", "keyboard emulation" or "plug-and-play" on the product page.

The scanner must support the two symbologies mybibli uses:

- **EAN-13** — the 13-digit code on the back cover of books, CDs, DVDs, and most consumer products. Universally supported.
- **Code-128** — the symbology mybibli uses to encode V-codes and L-codes on the labels it prints (see section 2.2). Universally supported.

You do *not* need a 2D imager (the kind that reads QR codes) — a basic 1D laser or LED scanner is enough. mybibli does not use QR codes.

Configuration

Three settings on the scanner matter. Most scanners ship them correctly out of the box; if your scans behave strangely, check the scanner's manual or the configuration sheet of barcodes that comes in the box:

- **Terminator / suffix:** `Enter` (also called `CR` or `Carriage Return`). mybibli's cataloging form auto-submits on Enter, so this turns each scan into one round-trip. Anything else (`Tab`, no terminator) will not work without manual key presses.
- **Prefix:** none. Some scanners can prepend a fixed string before the payload — leave it disabled.
- **Keyboard layout:** *must* match the layout of the host the browser runs on. If your laptop uses an AZERTY layout but the scanner is configured for QWERTY, the digits come out as letters (e.g. `1234567890` arrives as `&é"'(-è_çà`) and no ISBN ever resolves. The scanner's manual will have a code chart to switch.

Tip

USB scanners on a Synology NAS host: it does not matter that the scanner is plugged into the laptop you use to access the mybibli web UI rather than the NAS itself. The scanner is a keyboard for the browser, not for the server. The same setup works with a phone-side scanner app, an industrial Bluetooth scanner paired to the laptop, or any USB stick scanner.

What to do when the scanner is unavailable

Every scan field also accepts hand-typed input. Click into the field labelled "Scan ISBN" (or "Scan V-code" / "Scan L-code" elsewhere), type the digits, press Enter. The form behaves identically.

2.4 Reference data

Four taxonomy tables drive the dropdowns you see while cataloging. They live under `/admin > Reference data`:

Genres The genres assigned to titles. mybibli ships with a baseline FR/EN seed (Fiction, Non-fiction, Biography, ...) — edit or extend to match your collection.

Contributor roles Author, Illustrator, Translator, Editor, Narrator, ...Used by title detail pages to attribute the right people to the right slots.

Volume states The physical condition of an exemplaire — New, Good, Worn, Damaged, ...Useful for loan returns (mark a volume Worn when the borrower damaged it).

Location node types See section 2.1 above.

CRUD semantics

All four tables share the same CRUD pattern:

- **Create** adds a row. If the name collides with a soft-deleted row of the same table, the existing row is reactivated and the result reads *Reactivated* instead of *Created*.
- **Rename** updates the name. For location node types, the new name cascades to every location that uses the type.
- **Delete** performs a soft-delete *only when the row is unused*. If any title still references the genre / role / state / type, the deletion is refused with a count of dependents.

Reference data is not translated

Reference data values are stored verbatim — the FR seed creates a genre called “Roman” and that is the literal text English-speaking users will see if they switch the UI to English. Only the UI chrome (button labels, navigation, error messages) is translated.

2.5 System settings

The `/admin > System` tab groups every global setting. The panel is divided into three forms, each with its own save button:

Loans

- **Overdue threshold (days).** Defines how many days a loan can run before it surfaces as overdue on the dashboard. Default 30.
- **Session inactivity timeout (seconds).** Idle time after which an authenticated session is wiped. Default 1800 s (30 minutes).
- **Auto-purge interval (seconds).** How often the background scheduler hard-deletes items older than 30 days in the trash. Default 86400 s (24 hours).

Metadata providers

One row per keyed provider (Google Books, OMDb, TMDb). Each row has a key input and a *Clear* button. Saving a key takes effect on the next metadata fetch — no restart, no re-deploy.

Note

If the same provider key is set in `.env` *and* cleared from the UI, the next reboot re-migrates the value from `.env`. The operator’s deployment-time intent wins on boot. To make a

> Clear durable across reboots, also remove the variable from `.env` (or `docker-compose.yml`).

Language

Sets the default UI language for *anonymous* visitors. Authenticated users override this with the language toggle in the navigation bar — the choice is stored on their user row.

2.6 Users

Open `/admin > Users` to manage user accounts. Each row shows the username, role, active flag, and last-seen timestamp. The available actions are:

- **Create user** (top of the page) — username, password, role.
- **Edit** — change role or rename. Cannot change another user's password from this view; password resets go through the user's own `Account` page.
- **Deactivate** — soft-delete the user. Their sessions are wiped immediately; they cannot log back in.
- **Reactivate** — undo a deactivation (from the `Trash` tab).

Two safety guards apply:

- **Self-deactivate is blocked.** You cannot deactivate the account you are logged in as.
- **Last-active-admin guard.** The system always keeps at least one active admin. Deactivating the last one is refused with a 409 conflict and a clear error.

The three roles (Anonymous, Librarian, Admin) are detailed in chapter 6.

Chapter 3

Everyday use

This chapter walks through the workflows you will run dozens of times once the library is live: cataloging a new volume, searching your collection, moving a volume, lending it out, taking it back, and auditing what is on the shelves.

3.1 The home page

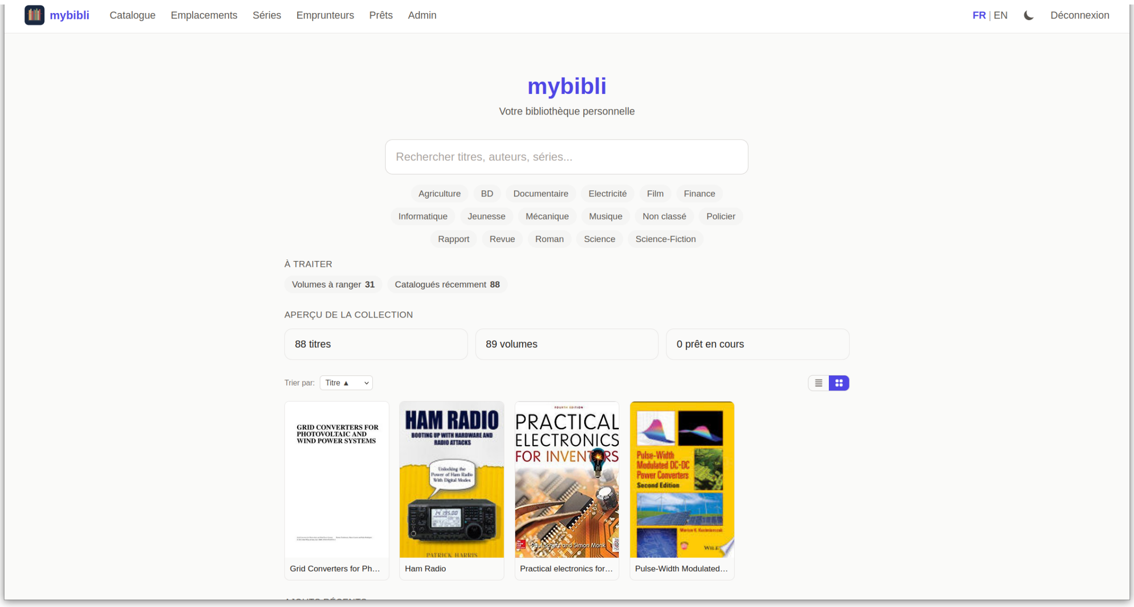

Figure 3.1: The home page on a live install: search bar, genre filter chips, “À traiter” counters, “Aperçu de la collection” totals, and a recent-additions strip with cover thumbnails fetched through the metadata-provider chain.

The home page is the dashboard. From top to bottom:

- **Scan field.** A focused input that accepts an ISBN, a V-code or an L-code. The scanner-guard layer routes the input to the right page depending on its prefix.
- **Indicators row.** Recent additions, overdue loans, series with gaps, unshelved volumes — each click jumps to the filtered list.
- **Recent activity.** The last seven days of cataloging and returns.

3.2 Cataloging a new title via barcode

1. Focus the home-page scan field (it is selected by default on page load). Scan the ISBN / EAN-13 of the back cover.
2. mybibli redirects to the catalog page, which displays a skeleton title with the barcode you scanned. A background task fires off requests against the metadata provider chain.
3. Within a few seconds, the skeleton fills in: title, contributors, publisher, year, cover image. A small indicator confirms which provider resolved the title.
4. Pick a **location** from the dropdown (or scan the L-code of the destination shelf) and pick a **volume state** (typically *New*). Click **Add volume**.
5. Print and stick a V-code label on the volume.

If the metadata pipeline fails to resolve the ISBN, the title appears with an *Unresolved* badge. You can edit the title fields manually and the volume is still created.

Tip

Cataloging in batches works best when the V-code sticker roll, the scanner and the printer are all within arm's reach. Most of the time is spent scanning, not waiting for mybibli — the metadata fetch is asynchronous, so you do not have to wait for it to finish before moving on to the next volume.

3.3 Searching the collection

The `/search` page accepts:

- full or partial **title**;
- **contributor name** (matches on author by default; other contributor roles are scored lower);
- **ISBN / EAN-13** — exact match;
- **V-code** or **L-code** — exact match;
- **Dewey code** (for libraries that use Dewey).

Results are sorted by relevance. Click a result to open the title page; click a contributor to filter the catalog by their work; click a location label to see every volume currently in that location.

Tip

You can "search" a single volume by scanning its V-code from any page that has the home-page scan field — that includes the home page, `/search` and `/loans`. Faster than typing.

3.4 Moving a volume to another location

1. Open the volume page (search by V-code or click through from a title page).
2. Click **Move**.

3. Scan the L-code of the destination location, or pick it from the dropdown.
4. Confirm.

mybibli records the move in the volume's location history so you can trace where a book has lived over time.

3.5 Loans

Lending a volume

1. Open `/loans` and click **New loan**.
2. Scan or pick the **borrower**. If the borrower does not exist yet, click *Create borrower* inline.
3. Scan or pick the **volume** (V-code).
4. Optionally set a custom **due date**; otherwise mybibli derives it from the overdue threshold (chapter 2).
5. Confirm.

The volume's current location is recorded on the loan row so a later return can restore it to the right shelf.

Returning a volume

1. Open `/loans` and find the active loan (filter on *Active* or scan the V-code).
2. Click **Return**.
3. Pick the post-return state (*Good*, *Worn*, …).
4. Confirm. The volume snaps back to its pre-loan location automatically.

Overdue loans

Loans that exceed the overdue threshold turn red on the `/loans` page and on the home-page *Overdue* indicator. You can run `/loans?filter=overdue` for a focused list.

3.6 Library audit

An *audit* is the routine "walk the shelves and confirm everything is where the database says it is". mybibli does not ship a dedicated audit mode — the equivalent workflow is:

1. Filter `/search` by the location you intend to audit, sorted by V-code.
2. Walk the shelf, scanning each V-code.
3. For any volume scanned that is *not* in the filtered list, mybibli surfaces a *Volume not in this location* message. Move it (section 3.4) and move on.
4. Any volume *in* the filtered list that you did *not* scan after walking the shelf is suspect — either misplaced or missing. Mark it for follow-up.

Tip

A quarterly audit catches misplaced volumes early — the longer a book has been on the wrong shelf, the harder it is to remember where it should be.

3.7 Shelving workflow

After a returns batch, the volumes pile up near the scanner station. mybibli's *shelving* flow speeds up putting them back:

1. Pick up a volume and scan its V-code.
2. mybibli shows the volume page with the original location prominently displayed.
3. Walk the volume to that shelf. Scan the L-code on the shelf to confirm.
4. Repeat.

Confirming with the L-code is optional but recommended — it leaves a fresh entry in the volume's location history that the next audit can rely on.

Chapter 4

Multiple media types

mybibli is not just for books. Four media types are first-class citizens:

- **Book** — the default. Single-volume novels, essays, reference works, children's picture books.
- **Comic / BD** — single-issue comics or multi-position omnibus albums (e.g. a "Tintin anthology" that physically gathers volumes 1–5).
- **Audio** — CDs, vinyl records, audiobooks on physical media.
- **Film / series** — DVDs, Blu-ray boxsets, TV-series collections.

The media type drives three things:

1. **Which metadata provider is consulted first.** ISBNs route to BnF + Google Books + Open Library; films route to TMDb + OMDb; audio routes to MusicBrainz.
2. **Which fields are exposed on the title page.** Audio titles surface a track list; films surface director, runtime, age rating.
3. **The icon shown next to the title** in search results and the catalog list.

4.1 Books

The default media type. Catalog as described in section 3.2. Multi-volume series (encyclopaedias, trilogies, …) are modeled with a *Series* entity in chapter 2.

4.2 Comics and BD

A single **omnibus** volume can contain several positions in a series. mybibli supports this with the *multi-position* feature: at volume creation time, you can set the volume's *position range* (for example, "contains volumes 5–7"). Series-gap detection (chapter 3) understands these ranges and will not flag positions 5, 6 or 7 as missing once the omnibus is on the shelf.

4.3 Audio releases

Audio titles use MusicBrainz as the primary metadata source. The barcode on the back of a CD (EAN-13) resolves the same way as an ISBN.

- **Tracks.** MusicBrainz returns the track list; mybibli renders it on the title page.
- **Format.** CD vs vinyl vs cassette is captured on the volume row (not on the title) — two identical pressings on different media are two volumes of the same title.

4.4 Films and series

Films and TV series use OMDb and TMDb as the primary metadata sources. A series boxset is modeled as a *title* with a multi-position volume covering the disc range.

Tip

OMDb is free for low volumes (under 1000 requests per day); TMDb is free with attribution. Either covers a household-sized DVD / Blu-ray shelf. See chapter 5 for setup.

4.5 Picking the media type at scan time

After you scan an unfamiliar barcode, mybibli guesses the media type from the EAN prefix (the GS1 country prefix and the resolved provider's response). The guess is shown on the catalog page; you can override it before clicking **Add volume** if it is wrong. Re-typing a title later is a few clicks (`/title/<id>`) and the provider chain re-runs against the new type if you trigger *Re-fetch metadata*.

Chapter 5

Metadata providers

When you scan an ISBN or an EAN-13, mybibli does not look the title up in a local catalog — it queries one or more external *metadata providers* until one returns a clean answer. This chapter explains the chain, where to get API keys, and how to rotate them safely.

5.1 The provider chain

mybibli walks the providers in the order best-suited to the guessed media type:

Provider	**Media types**	**Notes**	**Key?**
BnF	Books	French Bibliothèque nationale catalogue; best for French-language titles.	no
Open Library	Books	Volunteer catalogue; broad English / multilingual coverage.	no
Google Books	Books	Commercial; good for trade non-fiction.	optional
MusicBrainz	Audio	Volunteer music metadata; very deep on CDs / LPs.	no
OMDb	Films, series	Free tier; rate-limited.	required
TMDb	Films, series	Free with attribution.	required
BDGest	Comics, BD	FR-language comic catalogue (web-scraped).	no

The first provider to return a *clean* record wins. “Clean” means: title present, at least one contributor, and a year. Partial matches are recorded but mybibli keeps walking the chain until it finds a clean one or runs out of providers.

Cover-image fallback

When the resolving provider returns metadata but no cover URL (BnF never carries image URLs in its UNIMARC payload; Google Books occasionally lacks `imageLinks` for self-published titles) mybibli falls back to the *Open Library Covers API* (`https://covers.openlibrary.org/`) keyed by ISBN. That covers index is broad and independent of Open Library’s catalogue completeness, so most books still get a cover even when their text metadata comes from a different provider. The fallback runs only when (a) the chain succeeded *and* (b) the resolving provider gave no `cover_url`, so a title for which no provider has a cover anywhere will land cover-less without retry storms.

5.2 API keys

Three providers require an API key:

Google Books

Optional but raises the daily request budget significantly. Without a key the Google Books provider stays disabled — the chain still works (BnF → Open Library are key-less), but English-language trade non-fiction that BnF misses will land with empty metadata.

The Google Cloud Console flow trips up many first-timers because the API-key creation step is buried two menus deep. Walk through this exactly:

1. Open `https://console.cloud.google.com/` and sign in.
2. Top-bar project picker (next to the *Google Cloud* logo) → *New project* → name it anything (e.g. `mybibli-metadata`) → *Create*. Wait a few seconds for the project to be selected.
3. Open the hamburger menu (top-left, three horizontal lines) → *APIs & Services* → *Library*. Search for **Books API**, click it, then *Enable*.
4. Still under *APIs & Services*, open *Credentials*. Click *+ Create credentials* → *API key*. A modal pops up with a freshly generated key — copy it now.
5. Recommended: click *Restrict key* → under *API restrictions* pick *Restrict key* → tick only *Books API*. Leave *Application restrictions* on *None* (mybibli does not send a Referer header). Save.
6. In mybibli: `/admin > System > Metadata Providers` → paste into the **Google Books API key** field → *Save*. The next scan picks it up — no restart needed.

To verify the key is active, scan an English-only ISBN that BnF will not have (*Effective Java*, 9780134685991, is a good canary). The metadata should populate within a few seconds. If it stays empty, see *Troubleshooting* at the end of the chapter.

OMDb

1. Visit `https://www.omdbapi.com/apikey.aspx`.
2. Pick the free tier (1000 requests/day).
3. Confirm via the activation email you receive.
4. Paste the key into mybibli.

TMDb

1. Sign up at `https://www.themoviedb.org/`.
2. Open *Settings* → *API* and request a key for *Personal / non-commercial use*.
3. Paste the key into mybibli.

5.3 Where the keys live

Keys travel through two layers:

1. **Environment variables** (`GOOGLE_BOOKS_API_KEY`, `OMDB_API_KEY`, `TMDB_API_KEY`). These are migrated *once* on boot into the `settings` table — they are essentially a deployment-time bootstrap.
2. **The `settings` table**, exposed through `/admin > System > Metadata Providers`. This is the live source of truth at request time. Rotating a key from the UI takes effect on the very next metadata fetch.

The implication is asymmetrical:

- If you set a key only in the environment, the first boot copies it into the database and afterwards the database wins.
- If you set a key only in the UI, the environment is never consulted again.
- If you set both and then *Clear* the key from the UI, the database value goes empty — but on the next boot, the environment value is migrated back in. To make the UI clear durable, also remove the environment variable from `.env` / `docker-compose.yml`.

5.4 Provider health

`/admin > Health` pings every registered provider every five minutes and reports their status as a green check or a red cross. A red cross does not always mean the provider is down — it can also mean the API key is missing or invalid. The Health tab includes the last-checked timestamp and the HTTP status code for diagnostics.

5.5 Rate limits and back-pressure

mybibli applies a small per-provider rate limit (a couple of requests per second). Bulk-cataloging a hundred volumes in a row will not trigger remote 429 responses. If a provider does return 429, the request is logged and the next provider in the chain is consulted instead — the catalog flow never blocks waiting for a rate-limited provider.

Warning

Never commit a real API key to git. The `.env` file is in `.gitignore` for this reason; `.env.example` carries only placeholders. If a key leaks, rotate it at the provider console immediately — pasting a new key into mybibli takes effect within seconds.

5.6 Troubleshooting empty metadata

If a freshly scanned ISBN ends up with empty title / author / description fields, walk this list before suspecting a bug:

1. **Provider status in `/admin > Health`.** If Google Books or Open Library show a red cross, those providers will be skipped on every fetch. Re-check the last-checked HTTP code — `403` or `401` usually means a missing or invalid API key, not a remote outage.

2. **Language of the title.** BnF is the primary provider and is FR-leaning. For English-only books, the chain falls through to Google Books — which means the key (see above) is the unblocker.

3. **ISBN format.** mybibli normalises EAN-13 vs ISBN-10 internally, but some providers index only one of the two. A second scan a few minutes later sometimes hits a different fallback path.

4. **Manually edit and save.** You can always edit title, genre, contributors etc. by hand from the title's detail page — saving works even when no provider returned anything. The genre `Non classé` is the default for any title that came in without classification data.

If the fields are still empty after configuring the right keys and waiting a few seconds, open an issue on the project repo with the failing ISBN — the maintainers add provider mocks for reproducible cases.

Chapter 6

Roles and permissions

mybibli has three user roles ordered by privilege:

1. **Anonymous** — anyone who lands on the site without a session cookie. Read-only access to the catalog. Cannot loan, edit or delete anything.
2. **Librarian** — authenticated user whose role is `librarian`. Day-to-day cataloging, loans, returns, location moves. Cannot manage users or change system settings.
3. **Admin** — authenticated user whose role is `admin`. Everything a Librarian can do, plus the `/admin` panel: users, reference data, trash, system settings.

6.1 Who can do what

Action	**Anon**	**Lib**	**Admin**
Browse / search the catalog	yes	yes	yes
View title and volume pages	yes	yes	yes
View location tree	yes	yes	yes
Scan / catalog a new title	no	yes	yes
Add or remove a volume	no	yes	yes
Move a volume between locations	no	yes	yes
Edit a title's metadata	no	yes	yes
Create / edit borrowers	no	yes	yes
Register / return loans	no	yes	yes
Edit storage location tree	no	yes	yes
Open `/admin`	no	no	yes
Create / deactivate users	no	no	yes
Edit reference data	no	no	yes
Restore from trash	no	no	yes
Permanently delete from trash	no	no	yes
Edit system settings	no	no	yes

6.2 Sessions and the inactivity timeout

After login, mybibli sets a session cookie that travels with every subsequent request. The cookie is:

- `HttpOnly` — not exposed to JavaScript;
- `SameSite=Lax` — sent on top-level same-site navigations but not on cross-site requests;
- `Secure` (only when `MYBIBLI_COOKIE_SECURE` is set to `true` — see chapter 1).

The session has an **inactivity timeout** (default 30 minutes, configurable from `/admin > System > Loans`). Every authenticated request resets the countdown; if no request lands within the timeout window, the session is wiped server-side and the next request redirects to the login page.

A small *keep-alive toast* appears in the corner of the screen roughly two minutes before expiry — click it to extend the session without leaving your current page.

6.3 The last-active-admin guard

mybibli refuses to leave the installation without any active admin. The guard applies to two actions:

- **Deactivating a user.** If the user is the only active admin, the request returns a 409 conflict and the deactivation is refused.
- **Permanently deleting a user from the trash.** Same rule.

Self-deactivation is also blocked unconditionally (you cannot deactivate the account you are currently logged in as).

Warning

The guard counts only *active* admins. If you mass-deactivate admins and the guard fires, undo by reactivating someone from the trash tab. There is no recovery path through the wizard or the database without manual SQL surgery.

6.4 Password reset

mybibli does not implement password-reset emails — household / small-community deployments rarely benefit from the email plumbing. An admin can reset another user's password by:

1. Open `/admin > Users`.
2. Pick the user, click **Deactivate**.
3. Click **Reactivate** from the trash tab.
4. Re-create the user with a fresh password.

A user who knows their own password can change it from the `/account` page.

6.5 Anonymous sessions

Anonymous visitors also get a session row (used for CSRF protection and pending HTMX updates). Anonymous sessions expire after seven days of inactivity and are garbage-collected by a daily background task — they do not affect logged-in users.

Chapter 7

Backup and restore

A self-hosted library has no cloud safety net. This chapter covers what to back up, how often, and how to restore.

7.1 What to back up

Three artifacts together form a complete mybibli backup:

1. **The MariaDB database.** Holds every title, volume, location, contributor, loan, user, audit log and setting. Lose this and you start from zero.
2. **Cover images.** Downloaded during cataloging and persisted in the Docker volume `mybibli_covers` (or in your host bind-mount path if you switched per section 1.3). Lose this and the catalog still works, but every title page shows a generic placeholder until you re-fetch metadata title-by-title (issue #213 — pre-1.1.4 installs without this volume had covers disappear on every container upgrade).
3. **The `.env` file.** Holds passwords (database, API keys not yet migrated). Lose this and a re-deploy needs you to re-enter every credential.

The Docker images themselves are *not* backup-critical — you can always pull them again from Docker Hub.

7.2 Backup procedure

Database — bundled MariaDB

From the host running the compose stack:

```
docker compose exec -T db mariadb-dump \
    -u root -p"$MYSQL_ROOT_PASSWORD" \
    --single-transaction --routines --triggers \
    mybibli \
    > backups/mybibli-$(date +%Y%m%d-%H%M%S).sql
```

The `--single-transaction` flag is important: it dumps a consistent snapshot without locking the table for the duration of the dump. `--routines --triggers` preserves any stored routines (mybibli does not currently use any, but the flag is cheap and future-proof).

Compress the dump if disk space is a concern:

```
gzip backups/mybibli-*.sql
```

Database — external MariaDB

Run `mariadb-dump` directly against the external server. Synology DSM users can also schedule a *Hyper Backup* task that targets the MariaDB package — it produces a binary backup that restores faster than a SQL dump.

Covers and `.env`

For a host-mounted covers directory (bind-mount installs), a plain `tar` archive is enough:

```
tar czf backups/mybibli-files-$(date +%Y%m%d).tar.gz \
    .env "$COVERS_HOST_PATH"
```

For the default install (named Docker volume), back up the volume through Docker — the host path of a named volume varies by Docker version and is not stable, so don't try to `tar` it directly:

```
docker run --rm \
    -v mybibli_covers:/source:ro \
    -v "$(pwd)/backups:/backup" \
    alpine \
    tar czf "/backup/mybibli-covers-$(date +%Y%m%d).tar.gz" -C /source .
tar czf "backups/mybibli-env-$(date +%Y%m%d).tar.gz" .env
```

The throwaway `alpine` container mounts the named volume read-only, the host `backups/` directory writable, and writes the tarball. Same shape works for restoring — see the Restore section below.

7.3 Recommended cadence

- **Daily** — automated database dump. Even on a seldom-changed catalog, a daily dump means worst-case data loss is bounded to one day.
- **Weekly** — covers and `.env`. These change only when you catalog new titles or rotate keys; weekly is plenty.
- **Off-site copy** — at least monthly. Rsync the `backups/` directory to a remote NAS, a USB drive you swap in / out, or a cloud-storage bucket. A fire that consumes the home server should not also consume your backups.

Tip

A backup that has never been restored is not a backup. Once a quarter, restore your latest dump into a throwaway docker environment and verify the title and loan counts match what `/admin > Health` reports on production.

7.4 Restore procedure

Database

For the bundled-database setup:

```
docker compose stop mybibli
docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" \
    -e "DROP DATABASE mybibli; CREATE DATABASE mybibli CHARACTER SET utf8mb4;"
zcat backups/mybibli-20260513-1200.sql.gz | \
    docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli
docker compose start mybibli
```

For external MariaDB, the same `mariadb` client commands work against the external server.

Covers and `.env`

For a host-bind-mount install:

```
tar xzf backups/mybibli-files-20260513.tar.gz
```

For the default named-volume install, restore through a throwaway container in the symmetric shape to the backup:

```
docker compose stop mybibli
docker run --rm \
    -v mybibli_covers:/target \
    -v "$(pwd)/backups:/backup:ro" \
    alpine \
    sh -c "rm -rf /target/* && tar xzf /backup/mybibli-covers-20260513.tar.gz -C /target"
tar xzf backups/mybibli-env-20260513.tar.gz
docker compose start mybibli
```

Restart the stack so the mybibli container picks up the restored `.env`.

7.5 Migration to a new host

The backup procedure is also the migration procedure:

1. On the old host: produce a fresh database dump and tar archive.
2. Transfer both to the new host.
3. On the new host: install Docker, pull the compose file from the release matching the dump's version, place `.env` and `covers/` alongside, restore the database, start the stack.
4. Verify by counting titles on `/admin > Health` and spot-checking a few title pages.

Warning

The dump is tied to the *schema* version. If the source host ran 1.1.0 and the new host pulls 1.5.0, the new host's migrations will upgrade the restored database in place — that is fine. The reverse (restoring a 1.5.0 dump onto a 1.1.0 install) is *not* supported and will fail at boot. Always restore on a host running the same or newer mybibli release.

Chapter 8

Upgrade and migration

Routine upgrades are designed to be safe. The exception is the 1.0.x to 1.1.0 jump, which is a one-time breaking change documented below.

8.1 Routine upgrades (1.y to 1.z)

1. Read the release notes for every version between your current and the target version.
2. Take a fresh **database backup** (chapter 7).
3. Update the image tag in `docker-compose.yml` (or repull `:latest` if you track that tag).
4. Run `docker compose up -d`. The new container boots, runs any new migrations against the existing database, and starts serving requests.
5. Watch the logs for migration errors: `docker compose logs mybibli | grep -i migrate`.

Migrations are append-only — schema changes are forward compatible. A 1.4 install upgraded to 1.5 keeps every row from 1.4 and applies new tables / columns in place.

Tip

For homelab installs, the `:latest` Docker tag is a reasonable default. For production-style setups (a community library, an association), pin to an explicit version tag so an unattended pull does not bring in a release you have not read about.

8.2 The 1.0.x to 1.1.0 jump

Warning

This is a one-time breaking change. 1.0.x installs are subject to the issue documented in #173: the seed migrations create the credentials `admin/admin` and `librarian/librarian` on every fresh install, including production, and silently bypass the first-launch setup wizard.

The recommended path depends on whether your 1.0.x install holds real data:

If your 1.0.x install is empty (no titles cataloged)

The simplest path: wipe the database and reinstall on 1.1.0+.

1. Stop the stack: `docker compose down`.
2. Remove the database volume: `docker volume rm mybibli_mybibli_db_data` (or the equivalent volume name on your host — `docker volume ls` lists them).
3. Pull the 1.1.0 image: `docker compose pull`.
4. Update the image tag in `docker-compose.yml` if you pinned a previous version.
5. Start: `docker compose up -d`.
6. Open `/setup` and walk through the wizard.

If your 1.0.x install holds real data

1. Take a database backup (chapter 7).
2. Take a covers + `.env` backup.
3. Log in as the seeded admin and rotate the password *before* you upgrade. The 1.1.0 upgrade flow itself is safe, but rotation closes the only window where the default credentials could leak.
4. Update the image tag in `docker-compose.yml` and run `docker compose up -d`.
5. The 1.1.0 boot detects the existing admin and does *not* re-enable the wizard — the rotated admin remains the active admin.

The new `MYBIBLI_SEED_DEV_USERS` variable

Starting with 1.1.0, the seed migration is gated by `MYBIBLI_SEED_DEV_USERS`. The default is `0` (no seed). Only set it to `1` for local-development workflows or automated tests — never in production. The first-launch wizard becomes the canonical onboarding path.

8.3 Reading the release notes

Every mybibli release publishes a GitHub Release page with:

- a one-paragraph summary of the change set;
- the migration impact (none / additive / breaking);
- a link to the diff against the previous tag;
- attached PDFs of this manual (English + French) once the release-time PDF build is wired into CI.

Read the notes before upgrading. Breaking changes are tagged with a prominent *Breaking* label.

8.4 What's new in 1.1.7

Version 1.1.7 is a single-fix follow-up to 1.1.6 — same area (the Open Library Covers fallback introduced for issue #225), different code path. No schema migrations, no breaking changes — routine `docker compose pull && docker compose up -d`.

- **Re-fetch metadata now applies the cover fallback.** v1.1.6 wired the Open Library Covers fallback into the *background* metadata-fetch path that fires on a fresh ISBN scan, but missed the user-triggered *Re-fetch metadata* button on a title's detail page. Symmetric bug: FR titles resolved by BnF still landed cover-less on re-fetch even though Open Library may have had the cover indexed by ISBN. v1.1.7 extracts a single helper (`CoverService::resolve_cover_url_with_fallback`) used by every code path that downloads a cover for a resolved metadata result — background scan, direct-apply re-fetch (no manual edits), and the conflict-confirmation form's *accept cover* checkbox. To recover a previously cover-less title, re-open its detail page and click *Re-fetch metadata*; the cover now arrives in the same request when Open Library has it. Closes GH issue #228.

8.5 What's new in 1.1.6

Version 1.1.6 is a single-fix release on top of 1.1.5. No schema migrations, no breaking changes — routine `docker compose pull && docker compose up -d`.

- **Cover images for titles resolved by BnF.** The metadata-provider chain is "first-match-wins for the whole result". When BnF (the chain's primary FR book provider) resolved a title it would set the metadata fields but never a cover URL — UNIMARC XML, the format BnF returns via its SRU API, simply doesn't carry image URLs. The same happened for Google Books results without an `imageLinks` block (common for self-published Amazon KDP titles). The chain didn't fall through to Open Library for the cover, so most catalog entries landed with no jacket image even when Open Library had one.

  mybibli now performs a cover-only fallback: when the chain returns metadata but no `cover_url` and the lookup code is an ISBN, it HEAD-probes the *Open Library Covers API* (`https://covers.openlibrary.org/b/isbn/`). On a `200` the cover is downloaded, resized to 400 px wide, and saved as JPEG 80 % as if the original provider had supplied it. On `404` / network error / non-ISBN code the title stays cover-less, but quietly (no warn-log noise per missed cover).

  To recover covers on titles cataloged before this release, open each affected title's detail page and click *Re-fetch metadata*. A bulk-fetch admin action is tracked at issue #214. Closes GH issue #225.

8.6 What's new in 1.1.5

Version 1.1.5 continues the production-feedback patch series on top of 1.1.4. No schema migrations and no breaking changes — routine `docker compose pull && docker compose up -d`. Three improvements:

- **Save title with no metadata.** When the provider chain returns nothing for a scanned ISBN (BnF / Google Books / Open Library all silent), the title was created with the `Non classé` default genre but a manual save from the edit form could trip an FK violation if the submitted `genre_id` was 0 — a corner case that hit users editing freshly-scanned titles whose dropdown had no pre-selection. The handler now falls back to `Non classé` for any `genre_id == 0` input, and the underlying lookup self-heals (reactivates a soft-deleted row, re-seeds when missing). Closes GH issue #203.

- **Modal-Confirm header hardening.** The `ModalConfirmRetargetGuard` now requires `X-Modal-Confirm: true` AND `HX-Request: true` before it will strip `HX-Retarget` / `HX-Reswap` from a response. A non-HTMX client (curl, wget, scripted non-browser) can no longer trip the strip just by sending the modal header. Defense-in-depth — the threat model keeps this at low severity for the single-tenant LAN shape. Closes GH issue #218.

- **Google Books configuration walk-through.** The metadata-providers chapter (Chapter 5) now carries a step-by-step Google Cloud Console procedure for the Books API key, plus a *Troubleshooting empty metadata* checklist for when a scan lands with empty fields. Docs slice of GH issue #202; the broader per-provider visibility ideas (metadata-source badge, per-provider retry, structured error surfacing) remain open for future stories.

8.7 What's new in 1.1.4

Version 1.1.4 is a single-fix release. No schema migrations and no breaking changes for the *application* side — but the upgrade path required a `docker-compose.yml` change, which is why the cover persistence story is documented prominently in the Installation chapter rather than buried here.

- **Covers persist across container upgrades.** The published `docker-compose.yml` template now declares a `mybibli_covers` named volume mapped to `/app/covers`, parallel to `mybibli_db_data`. Pre-1.1.4 the cover JPGs lived inside the container's writable layer and were destroyed by every `docker compose up -d` after a `pull`. Existing users upgrading from a pre-1.1.4 install need to re-fetch covers from each affected title's detail page — a bulk-fetch admin action is tracked at GH issue #214. Closes GH issue #213.

8.8 What's new in 1.1.3

Version 1.1.3 is the first production-feedback-driven patch on top of 1.1.2. No schema migrations and no breaking changes — routine `docker compose pull && docker compose up -d`. Two user-visible improvements:

- **Home page reorder.** The filter results / search results area (the list that updates when you click a genre chip or type in the search box) now sits directly under the search box, ABOVE the indicator widgets (recent additions, recent returns, stats by genre, etc.). Pre-1.1.3 the results landed at the very bottom of the page — clicking a chip triggered the swap, but the swap happened five or more scroll-screens below the user's viewport, so it looked like nothing happened. Closes GH issue #204.

- **Locations tree rows are now clickable.** On the `/locations` page, clicking a location's name (or its icon, L-code, type label, or volume count) navigates to `/location/:id` — the per-location volume list with sortable columns (title, author, genre, Dewey). The edit/delete/add-child buttons stay individually clickable OUTSIDE the link. Closes GH issue #208.

8.9 What's new in 1.1.2

Version 1.1.2 closes Epic 10 — mobile UX and security closeout. There are no schema migrations and no breaking changes. The upgrade is a routine `docker compose pull && docker compose up`

`-d`. The user-visible improvements are:

- **Mobile data tables become cards.** The catalog, loans, borrowers, and series tables now collapse into a compact card list on narrow viewports (anything below the `md` Tailwind breakpoint). The desktop table layout is unchanged. The same rows are rendered in both shapes — the responsive switch is pure CSS, no JavaScript involved.
- **The admin tabs become a dropdown on mobile.** On narrow viewports, the five-tab admin nav (Health, Users, Reference data, Trash, System) renders as a `<select>` dropdown instead of overflowing horizontally. Deep links to `/admin?tab=...` keep working.
- **"Session expired" feedback is now explicit.** When the per-session CSRF token drifts (e.g. because the browser was idle past the inactivity timeout and the session rotated), the next state-changing action returns a clear server-rendered banner instead of a silent failure. The same flow handles session-timeout 403s.
- **Full accessibility coverage extended to entity pages.** The WCAG 2.2 AA gate in CI now covers every `/title/:id`, `/borrower/:id`, `/contributor/:id`, `/series/:id`, and the first-launch `/setup` wizard, not just the top-level navigation routes. Two contrast violations on the title detail page and the home recent-additions block were fixed inline.
- **Auth posture documented end-to-end.** `docs/auth-threat-model.md` now formalises the single-tenant LAN/NAS deployment shape: what CSRF protects, what session cookies do, why specific design choices were made, and what would need revisiting if multi-tenancy became a goal. Referenced from `README.md` and `CLAUDE.md`.

8.10 Rollback

Rollback is *not* a first-class flow. Schema migrations move forward only — there is no `down.sql`. The supported recovery strategy is:

1. Restore the pre-upgrade database backup (chapter 7).
2. Pin the image tag back to the prior version.
3. Start the stack.

This works because the backup is consistent with the prior schema. Trying to point an older image at a newly-migrated database will fail to boot.

Chapter 9

Best practices

Concentrated lessons from real-world deployments. Every section is optional — pick what fits your scale.

9.1 Storage hierarchy

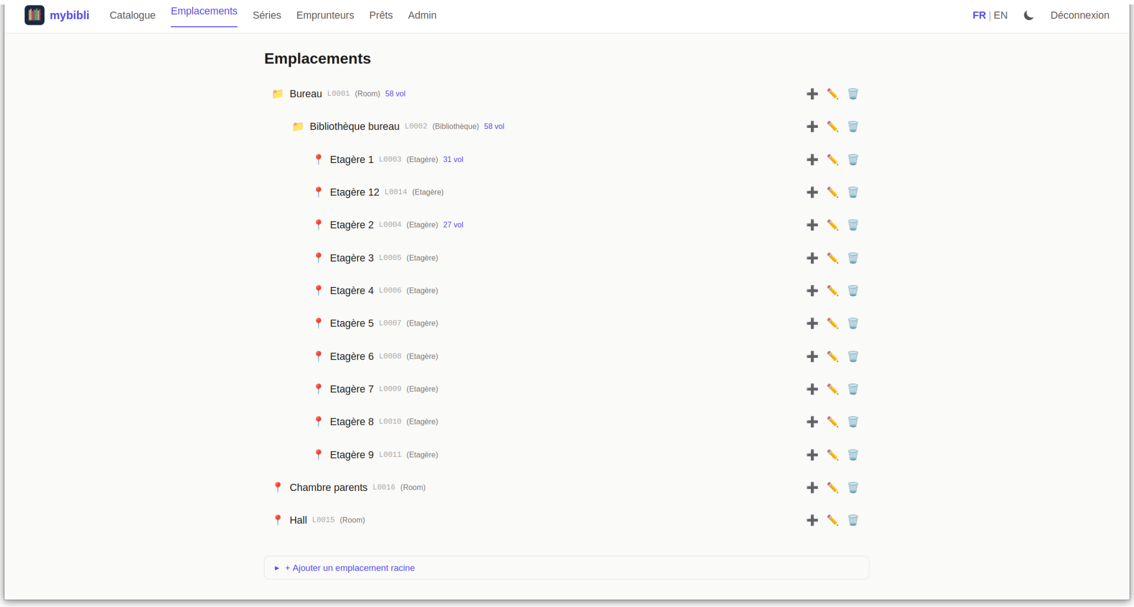

Figure 9.1: The Locations page on a live install: a household hierarchy with a study room, an in-room bookcase, ten shelves, and two flat-level rooms — each node carrying its L-code and a live volume count.

- **Three levels are usually enough.** Room → shelf → row covers most households. A fourth level (e.g. "box inside row") becomes paperwork you have to maintain.
- **Name locations the way you talk about them.** "Living-room shelf, top" beats "LR-S01-R01". The names are visible on barcode labels; treat them as user-facing.
- **Print L-codes once.** Re-printing because you rebalanced shelves is wasted effort — L-codes are stable identifiers; the human-readable name on the label can drift and that is fine.

9.2 V-code rolls

- Print V-codes in rolls of 50 to 100. Pre-printed stickers in a holder beside the scanner station make cataloging *fast*.
- **Stick the V-code before scanning.** It removes a future failure mode where you mislabel a book and have to scan-find it again.
- **Place the sticker consistently.** Inside back cover, upper-right corner, parallel to the spine — pick a rule and stick to it. Future-you doing an audit will thank present-you.

9.3 Genre versus Dewey

mybibli supports both broad genres and Dewey codes. Pick one as your primary navigation aid:

- **Use genres** when the audience browses by mood ("I want a thriller") and the collection is small enough that "Thriller" is granular enough.
- **Use Dewey** when the audience browses by topic ("books on 18th-century French history") or when the collection has reached the size where "Non-fiction" is no longer a useful filter.
- Using both is fine; mybibli does not force a choice.

9.4 Audit cadence

- **Quarterly** for collections under a few hundred volumes.
- **Monthly** for collections above that, especially if more than one person catalogs.
- Always run an audit *after* a major shelf rebalance — moving books physically without telling mybibli is the single biggest source of "where is this volume?" bugs.

9.5 User accounts

- **One human, one account.** Sharing a single `admin` account between two people loses the audit trail and makes the last-active-admin guard less useful.
- **Most editors should be Librarians, not Admins.** The admin panel handles installation-time concerns and rare cleanups; you do not need admin rights to catalog or loan.
- **Rotate the seeded admin password on first login.** On 1.1.0+ the setup wizard forces this; on legacy 1.0.x installs do it manually before exposing the instance to a network.

9.6 Loans

- **Keep borrowers identifiable.** "Marie L." beats "Marie" the moment the second Marie shows up. Email or phone in the notes field helps for chase-ups.
- **Run the overdue list weekly.** The dashboard indicator nags you for a reason — overdue loans compounded over a year become books you no longer own.
- **Mark damage at return time.** Switching a returned volume to *Worn* or *Damaged* captures the information while the borrower is in front of you; chasing the cost later from memory rarely works.

9.7 Metadata providers

- **Keep keys minimal.** You do not need all three keyed providers from day one — BnF + Open Library cover books for free. Add OMDb + TMDb the day you catalog your first DVD.
- **Treat keys as secrets.** `.env` is in `.gitignore`; keep backups of `.env` encrypted (chapter 7).

9.8 Upgrades

- **Read release notes before pulling `:latest`.** See chapter 8.
- **Back up immediately before any upgrade.** Even "additive" migrations have non-zero failure surface; a same-minute backup turns a worry into a chore.
- **Pin a version tag for production.** Auto-updates for a household instance are fine; for a community library, an unattended major bump can interrupt service.

Chapter 10

Troubleshooting

When things go wrong, start with the logs and the `/admin > Health` dashboard. This chapter catalogues the problems users actually hit, in rough order of frequency.

10.1 Reading the logs

```
docker compose logs --since 10m mybibli
docker compose logs -f mybibli
```

Logs are emitted as structured JSON. The most useful fields are `level`, `target` (which module logged this), `message`, and any contextual key=value pairs that the log line carries (`user_id`, `volume_id`, …).

To filter:

```
docker compose logs --since 10m mybibli | grep -i error
```

10.2 App does not start

Symptom: container exits immediately

Run `docker compose logs mybibli` and look for one of:

- `missing required environment variable: DATABASE_URL` — the `.env` file is missing or the variable is unset. See section 1.5.
- `Failed to create database pool` — the database hostname / port / credentials in `DATABASE_URL` are wrong, or the DB is not reachable. From the host, try `docker compose exec mybibli sh -c 'apk add curl; curl <db-host>:<db-port>'` to confirm reachability.
- `Failed to run database migrations` — usually a permissions issue. Confirm the DB user has `ALL PRIVILEGES` on the database (see section 1.4).
- `Address already in use` — the published host port is taken. Change `HOST_PORT` in `.env`.

Symptom: every page returns 404

The container is up but routes are missing. Most common cause: you pulled an image tag that does not exist (typo) and Docker fell back to a stale older tag. `docker compose pull` and restart.

10.3 Cannot log in

Symptom: "Invalid credentials" on a fresh 1.0.x install

The seed credentials are `admin/admin`. If they do not work either:

- the database migration that creates them failed silently — look for migration errors in the logs;
- the previous admin already rotated the password (check with whoever installed mybibli);
- you are looking at a 1.1.0+ install, where the seed credentials are planted by the migration but immediately soft-deleted by the issue-#173 seed gate at startup — use the wizard at `/setup` instead.

10.4 Stray users in the `users` table after the wizard

Symptom: four users in the database, only one was created in the wizard

If you inspect the `users` table directly (via phpMyAdmin or the MariaDB CLI) right after completing `/setup`, you will see *four* rows even though you only entered one username and password. This is by design. Three of the rows cannot be used to log in.

username	role	active	deleted_at
admin	admin	1	set, ≈ first-boot time
librarian	librarian	1	set, ≈ first-boot time
SYSTEM	system	0	NULL
<your-username>	admin	1	NULL

- `admin` and `librarian` are planted by the seed migrations (which run on every fresh install for the dev / E2E workflow) and immediately *soft-deleted* by the issue-#173 seed gate at startup. Their `deleted_at` is set, so the login query (which filters `deleted_at IS NULL`) refuses them.
- `SYSTEM` is a non-loginable utility account used to attribute the audit-log entries produced by the 30-day auto-purge background task. Its role is `system` (outside the login enum `admin/librarian`), its `password_hash` is the literal string `NO_LOGIN_SYSTEM_USER_HASH_NOT_A_VALID_ARGON2_` and its `active` flag is `0`.

Tip

To confirm the soft-deleted rows are inert, try logging in as `admin/admin` — it returns an authentication failure. The `/admin > Users` panel filters out both the `SYSTEM` row and any `deleted_at`-set row, so the admin UI shows the single account you created in the wizard.

The two soft-deleted rows are physically removed from the table by the 30-day auto-purge scheduler — expect them to disappear roughly one month after the boot date stamped in `deleted_at`.

Symptom: login redirects back to `/login`

The session cookie is being rejected by the browser. Causes:

- `MYBIBLI_COOKIE_SECURE=true` on a plain-HTTP deployment — the browser refuses to accept a Secure cookie over HTTP. Set the variable to `false` or front the instance with HTTPS.
- Cookies blocked by browser policy (private window with strict third-party blocking, etc.). Test in a default profile.

Symptom: locked out — last admin deactivated

The last-active-admin guard should prevent this, but if a previous admin was deactivated then permanently deleted, recovery requires direct SQL access:

```
docker compose exec db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli \
    -e "UPDATE users SET deleted_at=NULL, active=1 WHERE username='your-admin';"
```

Then log in normally.

10.5 Metadata never resolves

Symptom: skeleton title stays unresolved

Open `/admin > Health`. Look at each provider's status. A red cross usually means one of:

- the provider is genuinely down (rare for BnF / Open Library; more common for the smaller services);
- the API key is missing or invalid — open `/admin > System > Metadata Providers` and re-enter the key;
- the host cannot reach the upstream. Confirm with `docker compose exec mybibli sh -c "wget -q https://www.googleapis.com/books/v1 -O -"` or equivalent.

Symptom: only French ISBNs resolve

You disabled (or never set) the Google Books / Open Library keys. BnF is excellent for French-published titles but limited for foreign-language titles. Add at least Open Library (no key required).

10.6 Search returns too little

Symptom: known titles do not surface

- Confirm the title is not soft-deleted (check `/admin > Trash`). Search hides soft-deleted items.
- Confirm the title's contributors are spelled the way you searched. mybibli's search is fuzzy but not magical; "Camus" will find "Albert Camus" but "Camu" might not.

Symptom: search is slow

For collections above 10,000 titles, queries that combine contributor + free-text can take a second or two. This is a known limitation; planned indexes will close the gap.

10.7 Cover images are missing

Symptom: every title shows the placeholder

The cover directory is empty or unreadable. Confirm:

- `docker-compose.yml` declares the `mybibli_covers` named volume (or a bind mount on `COVERS_HOST_PATH`) mapped to `/app/covers`. Pre-1.1.4 installs did NOT declare this volume — see the next subsection;
- the metadata fetch actually returned a cover URL (check the title page — if the URL is missing, the provider did not supply one).

Symptom: covers vanished after upgrading the image

You upgraded from a pre-1.1.4 install and all previously cataloged covers are gone. This is issue #213 — the older compose template did not persist `/app/covers`, so the writable layer holding those JPGs was destroyed when the new container replaced the old one.

Fix forward: pull the 1.1.4 `docker-compose.yml`, run `docker compose down && docker compose up -d`. Subsequent covers are persisted in the `mybibli_covers` volume.

Recovery of lost covers: open each affected title's detail page and click *Re-fetch metadata* to repopulate from the provider chain. A bulk action is tracked at issue #214.

10.8 Where to file bugs

Issues you cannot resolve, or behaviour that contradicts this manual: open a ticket at `https://github.com/guycorbaz/mybibli/issues`. The issue template asks for the version (visible in the page footer), the install method, and the steps to reproduce — please fill them in.

Chapter 11

Glossary

Admin A user account with role `admin`. The only role that can open the `/admin` panel and edit users, reference data, trash or system settings.

Anonymous A visitor without a session cookie. Read-only access to the public catalog. No cataloging, no loans, no edits.

Audit The routine “walk the shelves and confirm everything is where the database says it is” workflow described in section 3.6.

BDGest A French-language comic / BD catalogue used as a metadata source for that media type.

BnF *Bibliothèque nationale de France*. The French national library catalogue, used as the primary metadata source for French-language books.

Borrower A person who loans volumes from the library. Modeled as its own entity (name, optional contact information, loan history).

Cataloging The act of registering a new title and / or volume in the database. Typically barcode-driven: scan, confirm, stick label, repeat.

Cover image The visual front of a book / disc / record, downloaded from a metadata provider and cached under `COVERS_DIR` on disk.

CSP Content Security Policy. A browser-enforced instructions header that restricts which scripts and styles the page may execute. mybibli ships a strict policy that forbids inline scripts and styles.

CSRF token A per-session secret embedded into every form. mybibli rejects POST / PUT / DELETE requests whose token does not match — a defense against cross-site request forgery.

Dewey code The Dewey Decimal Classification number, used for topic-based catalog navigation. Optional per title.

EAN-13 The 13-digit barcode standard. ISBNs are encoded as EAN-13 since 2007; CDs, DVDs and other media carry EAN-13 codes too.

Exemplaire Synonym for *volume* (the physical copy). Used by some French libraries; mybibli prefers *volume*.

Genre A free-form classification of a title's subject (Fiction, Non-fiction, Mystery, …). Defined in the genres reference table.

Hard-delete Deleting a row outright. mybibli only hard-deletes from the `/admin > Trash` panel and from the auto-purge background task; everything else is a soft-delete.

HTMX The JavaScript library mybibli uses to swap fragments of the page in place without full reloads. Operators do not need to understand HTMX to use mybibli.

ISBN International Standard Book Number. Identifier printed on the back cover of books, encoded as an EAN-13 barcode.

Last-active-admin guard The rule that prevents deactivating or permanently deleting the last user with the `admin` role. See section 2.6.

L-code A storage-location barcode label. Format: `L` followed by a zero-padded number.

Librarian A user account with role `librarian`. Can catalog, loan, return and edit most data but cannot administer users or settings.

Loan A record that a specific volume is currently out with a specific borrower. Tracks the loan date, due date, return date, and the volume's pre-loan location.

Location The physical place where a volume lives. Modeled as a tree (room → shelf → row), identified by an L-code.

MariaDB The database engine mybibli stores its data in. A fork of MySQL.

Media type The kind of object a title represents: Book, Comic / BD, Audio, Film / series. Drives provider selection and field display.

Multi-position volume A single physical volume that contains several positions in a series — typically an omnibus comic album.

MusicBrainz A volunteer-maintained music metadata catalogue, used as the primary source for audio media.

Node type A label applied to storage locations to indicate what kind of container they are (room, shelf, row, box). Pure label — no semantic enforcement.

OMDb Open Movie Database. Metadata source for films and TV series.

Open Library An open volunteer-driven catalogue of books, used as a metadata source.

Optimistic locking The technique mybibli uses to detect concurrent edits. Two users editing the same title get a clear error on the second save instead of silently overwriting each other.

Reactivation Restoring a soft-deleted row. For users, clears `deleted_at` and lets them log in again. For reference data, the term also covers re-creating a row with the same name as a soft-deleted one, which transparently undoes the delete.

Reference data The four taxonomy tables: genres, contributor roles, volume states, location node types. Editable from `/admin > Reference data`.

Series A named ordering of related titles (e.g. a trilogy or an open-ended comic series). Each title's position in the series is a small integer or a range (for omnibuses).

Setup wizard The four-step first-launch flow at `/setup` that an empty instance presents until an admin account exists.

Soft-delete Setting a row's `deleted_at` column to "now" instead of physically removing the row. Soft-deleted rows are invisible to normal queries but surface in `/admin > Trash` for 30 days, after which the auto-purge hard-deletes them.

Title The bibliographic entity (a book, a film, an album). A title can have multiple volumes — for example, two copies of the same novel on different shelves.

TMDb The Movie Database. A community-curated metadata source for films and TV series.

Trash The `/admin > Trash` tab. Lists soft-deleted entities of every type and lets an admin restore them or permanently delete them.

V-code A per-volume barcode label. Format: `V` followed by a zero-padded number.

Volume The physical object on a shelf — one copy of a title. A title can have many volumes; each carries its own V-code, location and state.

Volume state A reference-data row describing the condition of a volume: New, Good, Worn, Damaged, ….

Index